

Informe de Desarrollo — TP3

Creación del CRUD para administrar el Front-End

Proyecto	Pádel & Gol — Sistema de reserva de canchas deportivas
Materia	Aplicaciones Web II — Tecnicatura en Análisis de Sistemas
Stack	Node.js · Express · PostgreSQL · React
Equipo	Morán Santiago (Frontend / Testing) · Octavio (Backend / Integración)
Repositorio	github.com/santimoran19/PryMor-n-Fakiani

1. Descripción del proyecto

Pádel & Gol es una aplicación web full-stack orientada a la gestión y reserva de canchas deportivas (pádel y fútbol). El sistema permite a los usuarios consultar la disponibilidad de canchas y a los administradores gestionar el catálogo de las mismas mediante un panel dedicado. En esta tercera etapa (TP3) el foco estuvo en construir el backend completo que sustenta la administración del contenido, reemplazando la fuente de datos mockeada utilizada en el TP2 por una base de datos relacional persistente.

El objetivo de esta entrega fue implementar un CRUD completo sobre la entidad principal del dominio —las canchas— exponiéndolo a través de una API REST, y conectar dicho backend tanto con el sitio público (lectura) como con una interfaz de administración (lectura y escritura). Todo el ruteo se resolvió bajo un mismo dominio y puerto, tal como exige la consigna, organizando cuidadosamente los prefijos de cada conjunto de rutas.

2. Arquitectura del sistema

El proyecto se estructuró siguiendo el patrón **Modelo–Vista–Controlador (MVC)**, ampliado con una capa de servicios para desacoplar la lógica de negocio del manejo de las peticiones HTTP. Esta separación de responsabilidades favorece el mantenimiento, la escalabilidad y la legibilidad del código.

2.1. Capas de la aplicación

- **Modelo (Model):** encapsula el acceso a la base de datos PostgreSQL. Contiene las consultas SQL parametrizadas para operar sobre la tabla canchas, evitando inyección SQL mediante el uso de *placeholders* (\$1, \$2, ...).
- **Controlador (Controller):** recibe las peticiones, valida los parámetros de entrada, invoca a la capa de servicios/modelo y construye la respuesta HTTP con el código de estado correspondiente.
- **Servicios (Service):** aloja la lógica de negocio (validaciones de reglas, transformaciones de datos), manteniendo a los controladores delgados.
- **Vista / Presentación:** serialización de los datos a formato JSON para el cliente. El Front-End de administración (React) consume estos datos y renderiza la interfaz.

- **Rutas (Routes):** definen los *endpoints* y delegan en los controladores, diferenciando el ruteo de la API CRUD del de la API pública de solo lectura.

2.2. Estructura de directorios

```
src/
■■■ config/ → configuración y pool de conexión a PostgreSQL
■■■ models/ → acceso a datos (cancha.model.js)
■■■ controllers/ → lógica de petición/respuesta (cancha.controller.js)
■■■ services/ → lógica de negocio (cancha.service.js)
■■■ routes/ → definición de endpoints (cancha.routes.js, web.routes.js)
■■■ middlewares/ → validaciones y manejo de errores
■■■ app.js → punto de entrada y montaje de rutas
```

Este árbol modulariza el código por responsabilidad, lo que permite localizar y modificar funcionalidades sin afectar capas adyacentes, y deja preparado el proyecto para incorporar nuevas entidades (por ejemplo, reservas o usuarios en etapas posteriores) replicando el mismo esquema.

3. Endpoints de la API REST

Se implementaron dos conjuntos de rutas servidas bajo el mismo dominio y puerto. La **API CRUD** (prefijo `/api/canchas`) ofrece los cinco endpoints de administración exigidos —dos de lectura, alta, baja y modificación—, mientras que la **API pública de solo lectura** (prefijo `/api/web`) expone los datos al sitio para los visitantes.

Método	Ruta	Descripción	Respuesta
GET	<code>/api/canchas</code>	Lista todas las canchas registradas.	200 OK
GET	<code>/api/canchas/:id</code>	Obtiene el detalle de una cancha por su identificador.	200 / 404
POST	<code>/api/canchas</code>	Da de alta una nueva cancha (ALTA).	201 / 400
PUT	<code>/api/canchas/:id</code>	Modifica los datos de una cancha existente (MODIFICACIÓN).	200 / 404
DELETE	<code>/api/canchas/:id</code>	Elimina una cancha del sistema (BAJA).	204 / 404
GET	<code>/api/web/canchas</code>	API pública de solo lectura: listado para el sitio.	200 OK
GET	<code>/api/web/canchas/:id</code>	API pública de solo lectura: detalle para el sitio.	200 / 404

Todos los endpoints devuelven y reciben datos en formato JSON. Los códigos de estado HTTP se utilizan de manera semántica: 200 para lecturas exitosas, 201 para creaciones, 204 para bajas sin contenido de respuesta, 400 ante datos inválidos y 404 cuando el recurso no existe.

4. Base de datos

Se utilizó **PostgreSQL** como motor de base de datos relacional. La conexión se gestiona mediante un *pool* del paquete `pg`, cuyos parámetros (host, usuario, contraseña, base y puerto) se cargan desde variables de entorno para no exponer datos sensibles en el código fuente. La entidad central de esta etapa es la tabla `canchas`.

4.1. Estructura de la tabla `canchas`

Columna	Tipo	Restricciones	Descripción
<code>id</code>	<code>SERIAL</code>	<code>PRIMARY KEY</code>	Identificador único autoincremental.
<code>nombre</code>	<code>VARCHAR(100)</code>	<code>NOT NULL</code>	Nombre o denominación de la cancha.
<code>tipo</code>	<code>VARCHAR(30)</code>	<code>NOT NULL</code>	Disciplina: 'padel' o 'futbol'.
<code>descripcion</code>	<code>TEXT</code>	—	Detalle o características de la cancha.
<code>precio_hora</code>	<code>NUMERIC(10, 2)</code>	<code>NOT NULL</code>	Precio por hora de alquiler.
<code>imagen</code>	<code>VARCHAR(255)</code>	—	URL o ruta de la imagen ilustrativa.
<code>disponible</code>	<code>BOOLEAN</code>	<code>DEFAULT true</code>	Indica si la cancha está habilitada.
<code>creado_en</code>	<code>TIMESTAMP</code>	<code>DEFAULT now()</code>	Fecha y hora de alta del registro.

4.2. Script de creación (DDL)

```
CREATE TABLE canchas (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  tipo VARCHAR(30) NOT NULL,  
  descripcion TEXT,  
  precio_hora NUMERIC(10,2) NOT NULL,  
  imagen VARCHAR(255),  
  disponible BOOLEAN DEFAULT true,  
  creado_en TIMESTAMP DEFAULT now()  
);
```

5. Panel de administración

Se desarrolló una interfaz Front-End de administración que consume la API CRUD y permite gestionar el catálogo de canchas en su totalidad. Desde el panel, el administrador puede:

- Visualizar el listado completo de canchas en una tabla, con sus datos principales.
- Registrar una nueva cancha mediante un formulario validado (operación de ALTA).
- Editar los datos de una cancha existente, precargando el formulario (MODIFICACIÓN).
- Eliminar una cancha con confirmación previa (BAJA).
- Alternar la disponibilidad de cada cancha para habilitarla o deshabilitarla en el sitio público.

La comunicación con el backend se realiza mediante peticiones `fetch` asíncronas hacia los endpoints de `/api/canchas`. Tras cada operación de escritura, la vista se actualiza recargando el listado, garantizando consistencia entre la interfaz y el estado persistido en la base de datos. La validación se aplica tanto en el cliente (experiencia de usuario) como en el servidor (integridad de los datos), siguiendo el principio de no confiar únicamente en la validación del Front-End.

6. Conclusiones

El TP3 consolidó la transición de un backend que servía datos mockeados a una solución completa con persistencia real en PostgreSQL. La adopción del patrón MVC ampliado con una capa de servicios resultó fundamental para mantener el código organizado y desacoplado, facilitando el desarrollo paralelo entre las responsabilidades de backend e integración y las de frontend y testing.

La correcta diferenciación del ruteo entre la API CRUD y la API pública de solo lectura —ambas bajo el mismo dominio y puerto— permitió cumplir con la consigna sin generar conflictos de direccionamiento. Asimismo, el uso de consultas parametrizadas y variables de entorno sentó las bases de seguridad que se profundizarán en el TP4 con la autenticación, el hashing de credenciales y el control de acceso mediante JWT.

Como líneas de trabajo futuras se proyecta incorporar la gestión de reservas y usuarios, agregar paginación y filtros a los listados, e implementar el despliegue a producción contemplado como etapa opcional de la materia.